

Lab 8: BLE LED Control

Engineer: Stanley Etienne / J00974053

Course: ECEL-360

Date: 03-25-26

Objective

The objective of this lab was to configure the ESP32 as a Bluetooth Low Energy (BLE) server and use a smartphone application to wirelessly control the built-in LED. The ESP32 advertises itself as a BLE device, allowing a client device to connect and write data to a characteristic that controls the LED state (ON/OFF).

Materials

- ESP32 Development Board
- Micro-USB Cable
- Smartphone with BLE scanning app (To Connect)
- Arduino IDE

Background and Theory

Bluetooth Low Energy (BLE)

Bluetooth Low Energy (BLE) is a short-range wireless communication protocol designed for low power consumption and efficient data transfer. Unlike Wi-Fi, BLE does not require a router and allows direct communication between devices such as smartphones and embedded systems.

In BLE communication, devices take on specific roles:

- **Central (Client):** Initiates connection (smartphone)
- **Peripheral (Server):** Advertises and provides data (ESP32)

BLE communication uses the **Generic Attribute Profile (GATT)** to organize data:

- **Service:** A collection of related data
- **Characteristic:** A single data value inside a service

Each service and characteristic is identified using a **UUID**, which is a unique identifier used for communication between devices.

BLE Communication Flow

1. ESP32 advertises itself
2. Smartphone scans and detects device
3. Connection is established
4. Data is written to the characteristic
5. ESP32 responds by controlling the LED

Procedure

1. Opened Arduino IDE and included BLE libraries.
2. Initialized the ESP32 as a BLE device with a custom name.
3. Created a BLE server.
4. Defined a custom service using a UUID.
5. Created a writable characteristic to receive commands.
6. Programmed logic to interpret incoming BLE data ("ON"/"OFF").
7. Configured GPIO2 as the output for the built-in LED.
8. Uploaded the code to the ESP32.
9. Opened a BLE scanning app on the smartphone.
10. Connected to the ESP32 device.
11. Sent commands to toggle the LED ON and OFF.
12. Observed LED response and verified functionality.

Code:

```
/*
  Lab 8: BLE LED Control
*/

#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

#define LED_PIN 2

#define LED_SERVICE_UUID "levi"
#define LED_CHARACTERISTIC_UUID "umba"

BLECharacteristic *pCharacteristic;
bool deviceConnected = false;

// Handles connection events
class MyServerCallbacks : public BLEServerCallbacks {
  void onConnect(BLEServer* pServer) {
    deviceConnected = true;
    Serial.println("Phone connected.");
  }

  void onDisconnect(BLEServer* pServer) {
    deviceConnected = false;
    Serial.println("Phone disconnected.");
    pServer->getAdvertising()->start();
    Serial.println("Restarted advertising...");
  }
};

// Handles data written from the phone
class MyCallbacks : public BLECharacteristicCallbacks {
  void onWrite(BLECharacteristic *pCharacteristic) {
    String rxValue = pCharacteristic->getValue();

    if (rxValue.length() > 0) {
      Serial.print("Received: ");
      Serial.println(rxValue);

      rxValue.trim();
    }
  }
};
```

```

    rxValue.toUpperCase();

    // Active-low LED logic:
    // LOW = LED ON
    // HIGH = LED OFF

    if (rxValue == "OFF" || rxValue == "0") {
        digitalWrite(LED_PIN, HIGH);
        Serial.println("LED OFF");
        pCharacteristic->setValue("LED OFF");
        pCharacteristic->notify();
    }
    else if (rxValue == "ON" || rxValue == "1") {
        digitalWrite(LED_PIN, LOW);
        Serial.println("LED ON");
        pCharacteristic->setValue("LED ON");
        pCharacteristic->notify();
    }
    else {
        Serial.println("Invalid command. Send ON/OFF or 1/0.");
        pCharacteristic->setValue("Send ON/OFF or 1/0");
        pCharacteristic->notify();
    }
}
}
};

void setupBLE() {
    BLEDevice::init("SE_BLE_LED_Control");

    BLEServer *pServer = BLEDevice::createServer();
    pServer->setCallbacks(new MyServerCallbacks());

    BLEService *pService = pServer->createService(LED_SERVICE_UUID);

    pCharacteristic = pService->createCharacteristic(
        LED_CHARACTERISTIC_UUID,
        BLECharacteristic::PROPERTY_WRITE |
        BLECharacteristic::PROPERTY_READ |
        BLECharacteristic::PROPERTY_NOTIFY
    );

    pCharacteristic->addDescriptor(new BLE2902());
    pCharacteristic->setCallbacks(new MyCallbacks());
    pCharacteristic->setValue("Send ON/OFF or 1/0");
}

```

```

    pService->start();
    pServer->getAdvertising()->start();

    Serial.println("BLE LED Control is advertising...");
}

void setup() {
    Serial.begin(115200);

    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, HIGH);    // Start with LED OFF for active-low boards

    setupBLE();
}

void loop() {
    // Nothing needed here
}

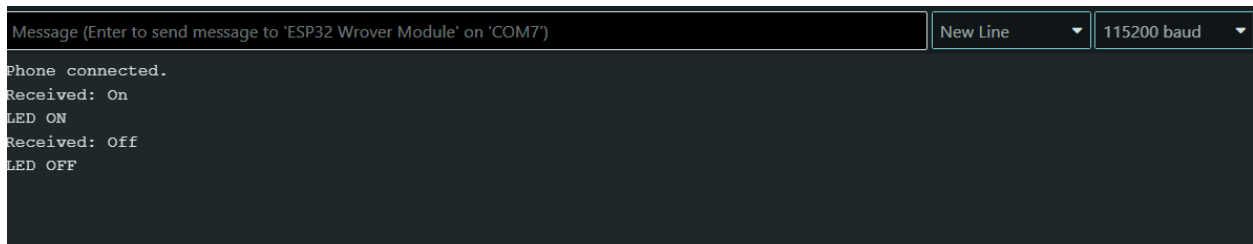
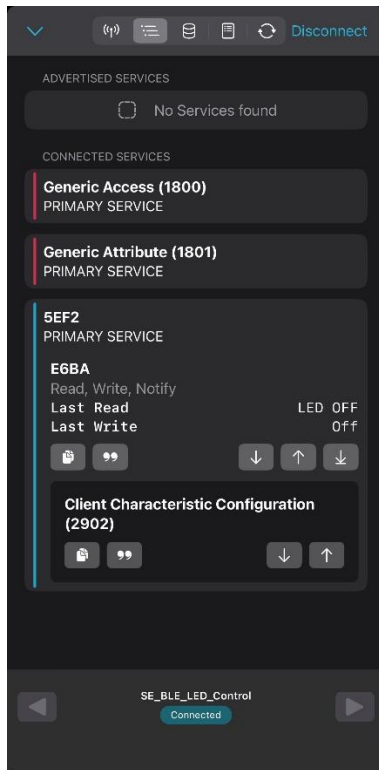
```

Results

The ESP32 successfully functioned as a BLE server and was detectable using a smartphone BLE scanning application. Once connected, the custom service and characteristic were visible within the app interface.

When the command **"ON"** was sent from the smartphone, the built-in LED turned on immediately. When the command **"OFF"** was sent, the LED turned off as expected. The system responded consistently to multiple command inputs without delay.

The Serial Monitor confirmed the received data, showing accurate communication between the smartphone and ESP32. Overall, the wireless control of the LED was stable and reliable.



Conclusion

This lab successfully demonstrated how to implement Bluetooth Low Energy (BLE) communication using the ESP32. The ESP32 was configured as a BLE server that advertised a custom service and characteristic, allowing a smartphone to connect and control hardware wirelessly.